ESPLORA TRAPANI EDU - Manuale tecnico di presa in carico

Architettura, infrastruttura, modello dati, contratti, sicurezza, operatività, test, estensione

Esplora Trapani · Alessandro Milia — redatto il 25 settembre 2026

0. Come leggere questo documento

Questo è il documento per **chi prende in mano il codice**. Presuppone familiarità con PHP, JavaScript vanilla, PostgreSQL e HTTP; non presuppone nulla del progetto. È organizzato così: prima il lessico e la mappa (§1-2), poi l'infrastruttura (§3), il modello dati e i contratti (§4-6), la sicurezza (§7), l'operatività (§8), i test (§9), e infine **come si aggiunge una funzionalità** con esempi guidati (§10). Chiude un elenco di trappole note (§11) e la mappa dei file (§12).

Convenzioni: `monospazio` = nomi di file, tabelle, funzioni, parametri. Le versioni citate sono quelle in produzione al 25/09/2026: **api.php v5 · player (index.php) v2.3 · regia v17 · live.php v1.2 · rispondi.php v1.1b · attiva.php v1.0 · stripe.php/stripe-webhook.php v1.0 · schema DB alla migrazione 30**.

1. Che cos'è, in tre paragrafi

Esplora Trapani Edu è una piattaforma educativa SaaS: lezioni in **video 360°** girate nei luoghi reali (saline, botteghe artigiane) che una classe vive sulla LIM dal browser, con punti interattivi ("hotspot") che aprono schede, una mascotte narrante, un quiz finale e un **quiz live** con PIN sui dispositivi degli alunni. Tutto è **accessibile** (modalità ipovedenti con lettura vocale, modalità sordi con interprete LIS sincronizzato), sempre incluso nel prezzo.

È **multi-cliente** ("multi-tenant"): ogni cliente (una salina, una scuola, un museo, un singolo docente) ha il suo spazio all'indirizzo `/project/edu/{slug-cliente}/`, con brand, corsi e licenza propri. I contenuti sono organizzati per **Cliente → Corso → Lezione**; chi li usa per **Cliente → Scuola → Classe**. Un corso esiste **una volta sola** e può essere **concesso in licenza** a più clienti ("catalogo", modello Netflix): chi lo riceve lo usa in sola lettura.

Il flusso di dati è: **Supabase** (PostgreSQL con Row Level Security) → **api.php** (guscio PHP su Aruba che protegge e cifra i contenuti) → **index.php** (il player, A-Frame + hls.js) sulla LIM. La **regia** (`regia-edu.php`) è il pannello di controllo, che parla direttamente con Supabase via

`supabase-js` con l'autenticazione dell'utente. **Stripe** gestisce gli abbonamenti e scrive la licenza tramite webhook.

2. Lessico

Termine	Significato nel sistema	Tabella
Cliente (tenant)	Chi ha lo spazio, il brand e la licenza. Slug nell'URL.	`edu_clienti`
Corso	Contenitore di lezioni con prologo, mascotte, stato (bozza/pubblicato/soon), **proprietario** (un cliente). Unica parola per "progetto/percorso".	`edu_corsi`
Lezione	Playlist di clip 360° + tappe + hotspot + quiz. Stato bozza/soon/pubblicata.	`edu_lezioni` + `edu_media`, `edu_tappe`, `edu_hotspot`, `edu_quiz`
Catalogo	Licenza d'uso di un corso a un cliente (stato + date). Il proprietario ha sempre una riga propria (backfill).	`edu_catalogo`
Scuola	Istituto che usa i corsi di un cliente: *ospite* (invitata) o *pagante* (licenza propria con date).	`edu_scuole`
Classe	Codice (es. `ICPA-5A-X1`) che identifica una LIM/classe nel tracciamento; appartiene a una scuola. Link: `/edu/{slug}/?classe=CODICE`.	`edu_classi`
Licenza a tre livelli	Una lezione si apre solo se: account cliente attivo **e** corso in licenza valida **e** scuola della classe entro le date.	funzioni `edu_licenza_attiva`, `edu_catalogo_valido`, `edu_scuola_valida`
Piano	base / pro / premium con limiti (`edu_piano_limiti`): corsi pubblicati, classi, quiz live.	colonna `piano`
Abbonamento	Specchio dello stato Stripe (`nessuno`, `in_attesa`, `attivo`, `in_pausa`, `insoluto`, `annullato`, `scaduto`).	colonne `abbonamento_*`, `edu_pagamenti`
Mascotte	Personaggio del corso: nome, immagine, fasce, video per 5 momenti × 3 fasce (jsonb sul corso). Il campo storico delle tappe si chiama ancora `cocciu`.	`edu_corsi.mascotte`

Termine	Significato nel sistema	Tabella
Fasce	`junior` (8-10) e `senior` (11-13): testi doppi (`txt/txt2`, `cocciu/cocciu2`), quiz doppi (`quiz/quiz2`).	—
Ruoli regia	`admin` (tutto, `cliente_id` NULL), `editor` (scrive nel proprio cliente), `docente` (legge il proprio cliente; gestisce solo le proprie classi).	`edu_utenti`
Progetto (legacy)	Vecchia partizione pre-SaaS (`sale-in-classe`, `antichi-mestieri-trapani`). Resta come colonna e come `progetto_legacy` sul cliente; non usarla per logica nuova.	—

3. Infrastruttura

3.1 Hosting Aruba (PHP)

- **PHP 8.0.30, LiteSpeed, suPHP**: file `644`, cartelle `755` — permessi `666/777` producono **500**. `exec()` disabilitato. `mbstring` non garantito (in `live.php` ci sono helper `taglia()`/`minuscolo()`).
- Percorso: `/project/edu/app/` (tutto il codice) e `/project/edu/.htaccess` (riscrittura dei tenant). Le cartelle `mestieri/ver7`, `mestieri/ver8`, `saline/` sono residui pre-SaaS da rimuovere quando `app/` è confermata.
- `.htaccess` (`htaccess-project-edu.txt`): forza la barra finale (`/edu/salina-culcasi` → 301 → `/edu/salina-culcasi/`), riscrive `/edu/{slug}/` → `app/index.php?c={slug}` e `/edu/{slug}/{file}` → `app/{file}?c={slug}` (QSA). Esclusi: `app`, `mestieri`, `saline`, `ver*`. Senza barra finale il browser risolveva `api.php` un livello sopra: per questo `index.php` espone `window.EDU_BASE` (percorso assoluto di `app/`) e tutte le chiamate lo usano.
- `app/dati/` è **negata dal web** (`dati/.htaccess`) e deve essere scrivibile da PHP: contiene `config.php` (segreti), `cache_*.json` (cache per cliente, TTL 5 min, quella del catalogo è anche **per giorno** perché le licenze scadono per data), `log.txt`, `rl/` (contatori del rate limit per IP), `live/{pin}.json` (stanze del quiz live).
- Host canonico: `index.php`, `rispondi.php`, `regia-edu.php`, `attiva.php` redirigono `alessandromilia.it` → `www.alessandromilia.it` (altrimenti il preflight CORS con header custom muore sul 301 del server).
- Diagnostica: `debug500.php?k=salina` (versione PHP, permessi, md5 normalizzato dei file, include del config). **Da rimuovere** in produzione stabile. `api.php?ver` stampa versione e tenant di default.

3.2 Supabase (progetto “ASK Trapani”, `sdqzqzmkhbkkibjzauox`)

- PostgreSQL + PostgREST + Auth. Tutte le tabelle hanno prefisso `edu_` e convivono con altri progetti (ASK Trapani, Welcome SaaS): **non toccare le altre**.
- Il player **non legge mai tabelle**: usa solo RPC `SECURITY DEFINER` con la chiave `anon` (§5). La regia usa `supabase-js` con l’utente autenticato e le policy RLS (§7).
- Chiavi: `SUPA_ANON` (pubblica, nel config e nella regia), `SUPA_SERVICE` (**service_role**, solo in `dati/config.php`, usata da `stripe.php` e dal webhook — non deve mai arrivare nel browser).
- Auth: utenti con email/password; `edu_utenti.id = auth.users.id`. `edu_utenti` **non ha la colonna email** (sta in `auth.users`). URL Configuration: Site URL e Redirect URLs (`https://www.alessandromilia.it/project/edu/**`) per il recupero password.

3.3 Bunny Stream (video)

- Libreria `esplora-trapani` (ID 721728), CDN `https://vz-bedaaab5-085.b-cdn.net/{guid}/` → `playlist.m3u8` (HLS), `play_720p.mp4`, `play_1080p.mp4`, `captions/it.vtt`, `thumbnail.jpg`. **Mai creare librerie nuove** (i GUID cambiano). Se «Allowed referrers» è impostato, tutto ciò che riproduce i video deve stare su `alessandromilia.it`.
- La pagina invia `Referrer-Policy: strict-origin-when-cross-origin` (con `same-origin` Bunny risponde 403).
- Audio di sottofondo per clip: file `.mp3/.m4a` su Bunny Storage o Backblaze B2, URL https diretto in `edu_media.audio_url`.

3.4 Stripe (abbonamenti)

- Modalità test e live con chiavi e price ID distinti. Prodotti: Base/Pro/Premium × mese/anno (creati da `stripe-setup.php`). Abbonamenti in modalità **`send_invoice`**: Stripe invia l’email con il link di pagamento; i rinnovi arrivano allo stesso modo.
- Webhook: `https://www.alessandromilia.it/project/edu/app/stripe-webhook.php`, eventi `invoice.paid`, `invoice.payment_failed`, `customer.subscription.{created,updated,deleted,paused,resumed}`.

4. Componenti e responsabilità

File	Ruolo	Note chiave
`index.php`	**Player** per la LIM. Bootstrap PHP (sessione, tenant, brand) + app JS (A-Frame, hls.js, TTS, LIS, quiz, audio clip, prologo, mascotte).	Zero contenuti nel sorgente: tutto arriva da `api.php` cifrato.
`api.php`	**Guscio**: token di sessione, referer, throttle	Azioni: `menu`, `lezione`, `lez`

File	Ruolo	Note chiave
	per IP, tenant, licenza (3 livelli), piani, cache per cliente, cifratura AES-256-CBC delle risposte, tracciamento.	(anteprima per token), `track`, `?ver`, `diag`.
`regia-edu.php` / `.html`	**Regia** (pannello). Le due varianti sono identiche salvo le 2 righe di config (PHP vs inline).	Supabase Auth + supabase-js; recupero password; 9 pannelli (§4.2).
`live.php`	**Quiz live** backend su file (`dati/live/{pin}.json`, `flock`).	Azioni: `crea`, `join`, `stato`, `risposta`, `avanza`, `rimuovi`, `chiudi`, `ver`. Throttle per azione.
`live-lim.js`	Modulo LIM del quiz live (lobby, QR inlinato, istogramma, podio), espone `EduLive.apri(cfg)`.	Caricato da `index.php` con percorso assoluto.
`rispondi.php`	Pagina alunno (mobile-first, orizzontale, vibrazione).	Polling di `stato` ogni ~2 s.
`attiva.php`	Pagina pubblica di richiesta attivazione → `edu_richieste` (anon insert-only, honeypot).	Le richieste compaiono in regia (Clienti → Richieste).
`stripe.php`	Comandi del manager verso Stripe (crea/pausa/riprendi/annulla/stato/link).	Solo admin (token Supabase verificato server-side).
`stripe-webhook.php`	Riceve eventi Stripe con firma HMAC → RPC `edu_abbonamento_evento` (chiave service).	Idempotente; 500 = “ritenta”.
`stripe-setup.php`	Crea prodotti/prezzi una volta e stampa `$STRIPE_PRICES`.	Protetto da `EDU_DIAG_KEY`; cancellare dopo.
`debug500.php`	Diagnostica del deploy.	Cancellare dopo.
`dati/config.php`	Tutti i segreti e le impostazioni (§4.1).	Negato dal web. Non sovrascrivere con lo starter dello zip.

4.1 `dati/config.php` — variabili

Variabile	Uso
`$SUPA_URL`, `$SUPA_ANON`	Endpoint e chiave pubblica Supabase (api.php, regia, attiva.php).
`$SUPA_SERVICE`	Chiave service_role: **solo** stripe.php e webhook.
`$EDU_TENANT_DEFAULT`	Tenant quando l'URL non ne contiene uno (`esplora-trapani`).
`$EDU_PROGETTO`	Legacy: progetto usato dal percorso senza tenant e dalla mascotte globale.
`$EDU_BRAND`, `$EDU_MASCOTTE`, `$EDU_MASCOTTE_QUIZ_SEMPRE`	Default globali del player (sovrascritti dal cliente/corso).
`$EDU_CACHE_TTL`	Secondi di cache dei JSON per cliente (300).
`$EDU_RATE_MAX`, `$EDU_RATE_WIN`	Rate limit per **sessione** (storico); quello per **IP** è in `edu_throttle()` (300/min).
`$EDU_CHECK_REFERER`	Controllo del referer sulle chiamate API.
`$EDU_DIAG_KEY`	Chiave per `api.php?action=diag`, `debug500`, `stripe-setup`.
`$STRIPE_SECRET`, `$STRIPE_WEBHOOK_SECRET`, `$STRIPE_PRICES[piano][mese\|anno]`, `$STRIPE_GIORNI_FATTURA`	Stripe.
`$STRIPE_API` (opzionale)	Base URL Stripe: si sovrascrive **solo nei test** per puntare al mock.

4.2 Regia: pannelli e funzioni principali

- **Lezioni** (sidebar raggruppata Cliente → Corso → Lezione, filtro testuale): editor piatto equirettangolare (canvas 2D — la videosfera A-Frame sul PC dell'autore restava nera per un overflow di texture HLS), playlist media con campi 🤟 LIS e 🎵 audio (URL, volume, ascolta), tappe con hotspot (yaw/pitch cliccando sul frame), quiz a due livelli con «Lo sapevi?» e video di riferimento `v`, accessibilità (template ipovedenti, conteggio clip LIS), versioni. Funzioni: `apriLezione`, `salvaMeta`, `renderMedia`, `caricaMediaNelViewer`, `vidStato`, `salvaLis`, `salvaAudio`.

- 🏢 **Clienti & Licenze** (admin): CRUD cliente, piano, stato/date licenza, referente, utenti (ruolo + cliente), 💳 **Abbonamento** (Stripe), 📥 Richieste da attiva.php con «Crea cliente». Funzioni: `apriCliente`, `salvaCliente`, `salvaUtente`, `abbonamento*`, `attivaDaRichiesta`.
- 📚 **Corsi**: metadati, prologo, mascotte (griglia 5×3 + importa da progetto legacy), visibilità (privato/catalogo), stato con limite del piano (`corsoPubblicabile`). Funzioni: `apriCorso`, `salvaCorso`, `nuovoCorso`, `renderMascotteCorso`, `mascotteCorsoPayload`.
- 🎞️ **Catalogo**: licenze del corso ad altri clienti (concedi/revoca/scadenza), **duplica** (`edu_duplica_corso`).
- 🏫 **Scuole e classi**: CRUD scuole (tipo, licenza), classi con codice generato (`codiceClasse`) e link del player.
- 📊 **Statistiche**: `edu_stats_cliente` → KPI, per scuola/classe/lezione, barre 30 gg.
- 🎭 **Mascotte & progetti**: configurazione legacy per progetto (sorgente per l'import).
- Trasversali: `esc()` (scappa anche apici e virgolette: i nomi finiscono negli `onclick`), `toast`, `nascondiTutto`/`chiudiPannello`, `caricaTenant` (carica `CLIENTI`, `CORSI`), `SLUG_RISERVATI`/`slugOk`, login con «Password dimenticata?» (Supabase recovery → form nuova password, messaggio se il link è scaduto).

5. Modello dati (schema alla migrazione 30)

5.1 Tabelle

Contenuti - `edu_lezioni(id, slug, titolo, sottotitolo, emoji, colore, durata_label, intro, ordine, stato, owner, origine_id, token, progetto, video_coccio, accessibilita jsonb, prologo, corso_id, creato_il, aggiornato_il)` — `owner` non nullo = copia personale di un docente; `token` = anteprima privata (`api.php?action=lez&t=`). - `edu_media(id, lezione_id, ordine, tipo, guid, url, durata_s, note, video_lis, audio_url, audio_vol, audio_loop, audio_duck)`. - `edu_tappe(id, lezione_id, media_id, t, ordine, cocciu, cocciu2)` — `t` secondi nella clip; `cocciu` = frase della mascotte. - `edu_hotspot(id, tappa_id, yaw, pitch, em, tit, txt, txt2, foto_url, audio_url, ordine, validazione)`. - `edu_quiz(id, lezione_id, livello, domanda, opzioni jsonb, ok, ordine, spiegazione, media_spiegazione, v)` — `v` = indice della clip (quiz progressivo). - `edu_versioni`, `edu_messaggi`, `edu_progetti`, `edu_mascotte` — legacy/servizio.

Tenant - `edu_clienti(id, slug, nome, brand jsonb, piano, licenza_stato, licenza_inizio, licenza_fine, contatto_nome, contatto_email, note, progetto_legacy, stripe_customer_id, stripe_subscription_id, abbonamento_stato, abbonamento_piano, abbonamento_periodo, abbonamento_fine, abbonamento_annulla_a_fine, abbonamento_link, abbonamento_aggiornato)`. - `edu_corsi(id, proprietario_id, slug, titolo, sottotitolo, descrizione, emoji, colore, copertina_url, fascia, prologo, stato, ordine, visibilita, mascotte jsonb, copia_di)` — `unique(proprietario_id, slug)`. - `edu_catalogo(id, corso_id, cliente_id, stato, da, a, note)` — `unique(corso_id, cliente_id)`. - `edu_scuole(id, cliente_id, nome, citta, referente, email, tipo, licenza_stato, licenza_da, licenza_a, note)`. - `edu_classi(codice PK, istituto, comune, livello, anno_scolastico, progetto, note, id, cliente_id, scuola_id, nome, anno, docente_id, creato_il)` — le prime colonne sono dell'agosto 2026 e restano per le viste `edu_v_report_*`. - `edu_utenti(id,`

`ruolo, istituto, nome, creato_il, cliente_id)`. - `edu_eventi(id, codice_classe, progetto, lezione, tipo, dettaglio jsonb, ts, creato_il, cliente_id, corso_id)` — la data storica è `ts`; `creato_il` è stata aggiunta (19b) e alimentata. - `edu_richieste`, `edu_pagamenti`.

5.2 RPC (tutte `SECURITY DEFINER`, `search_path = public`)

Funzione	Chi	Cosa
`edu_get_cliente(slug)`	anon	Cliente pubblico: brand, piano, `limiti`, `attiva`, licenza.
`edu_get_corsi(slug)`	anon	Corsi visibili al cliente (propri + in licenza valida), con `origine`, `licenza_a`, `mascotte`, lezioni non-bozza senza owner. Ordinati per `ordine` numerico.
`edu_get_lezione_tenant(slug, lezione)`	anon	Lezione completa (`edu_assembla_lezione`) + `corso{id,uuid,tit,prologo,mascotte}` solo se corso **pubblicato** e licenza valida.
`edu_classe_stato(cliente, codice)`	anon	`{valida, scuola, tipo, licenza_stato, licenza_a}`; classe non registrata = ammessa.
`edu_piano_limiti(piano)`	anon	Limiti del piano.
`edu_assembla_lezione(uuid)`	interna	Il JSON della lezione (§6.2).
`edu_e_admin()`, `edu_mio_cliente()`, `edu_sono_editor()`, `edu_sono_proprietario(corso)`, `edu_ho_licenza(corso)`, `edu_lezione_mia(lez)`, `edu_lezione_scrivibile(lez)`, `edu_corso_scrivibile(corso)`	authenticated	Helper delle policy: leggono le tabelle **senza** riattivare le policy del chiamante (evitano la ricorsione corsi↔catalogo).
`edu_duplica_corso(corso, cliente_dest, slug)`	authenticated (admin o editor del destinatario)	Copia corso + lezioni + media + tappe + hotspot + quiz in bozza, `copia_di` = originale.
`edu_stats_cliente(slug)`	authenticated	Totali, per_scuola, per_classe,

Funzione	Chi	Cosa
	(admin o proprio cliente)	per_lezione, ultimi_30gg. Media = `pct_classe` (live) o `punti/su` (classico).
`edu_abbonamento_cliente(cliente)`	authenticated (solo admin, altrimenti null)	Stato abbonamento + ultimi pagamenti.
`edu_abbonamento_evento(jsonb)`	**service_role**	Applica un evento Stripe: storico idempotente + stato + licenza.
`edu_get_lezioni(progetto)`, `edu_get_mascotte(progetto)`, `edu_get_lezione_token(token)`	legacy	Percorso senza tenant e anteprime.

5.3 Migrazioni

- Versionate in `migrazione_NN_*.sql` (19 → 19b → 20 … 30); **`migrazione_completa.sql`** le contiene tutte in ordine ed è **idempotente** (provata 3× di fila su una replica esatta della produzione). Supabase esegue un file come **blocco unico**: al primo errore annulla tutto.
- Regola: prima di scrivere una migrazione, esportare lo schema reale (`information_schema.columns` per `edu_%`), costruire la replica (`mock/replica.sql` nella cartella dei test), provarci sopra, poi consegnare.
- La 19b adatta le tabelle preesistenti (colonne mancanti, travaso date). La 26 rinomina `cliente_id→proprietario_id` sui corsi solo se serve (la 20 la crea già così). La 29 **azzera tutte le policy `edu_`** e ricrea il set canonico (§7). La 30 aggiunge gli abbonamenti.

6. Contratti (API e JSON)

6.1 `api.php`

Tutte le chiamate: `GET/POST api.php?action=…&c={slug}[&classe=CODICE]` con header `X-EDU-T: <token>` (token di sessione emesso in `index.php`, cookie PHP) e referer del sito. Risposte **cifrate**: `{enc:true, iv:base64, data:base64}` — AES-256-CBC con la chiave di sessione `EDU_K` (esposta solo nella pagina del player); il player decifra con WebCrypto (`eduApi`). Errori in chiaro: `{ok:false, err:'token'|'rate'|'referer'|'licenza'|'licenza_scuola'|'cliente'|'supabase'}` con 403/429/404/503.

- `action=menu` → `{ok, tenant, cliente: {id,slug,nome,brand,piano,limiti,licenza_stato,licenza_fine,attiva,contatto_email},`

`scuola:{nome,classe}|null, corsi:[…], lezioni:[…piatte], mascotte, mascotteLiv, mascotteCfg, quizSempre}`. Se l'account è scaduto: `{ok:false, err:'licenza', cliente}`; se la scuola della classe è scaduta: `{ok:false, err:'licenza_scuola', cliente, scuola:{nome,licenza_stato,licenza_a}}`.

- `action=lezione&id={slug}` → `{ok, lezione: <JSON §6.2>}`.
- `action=lez&t={token}` → anteprima di una lezione privata (bozza/copia).
- `action=track` (POST JSON `{classe, lezione, tipo, dettaglio}`) → inserisce in `edu_eventi` con `cliente_id`/`corso_id`; risponde sempre `{ok}`.
- Cache: `dati/cache_cliente_{slug}.json`, `cache_catalogo_{slug}_{Ymd}.json`, `cache_lez_{slug}_{lezione}.json` (TTL 300 s). Dopo una modifica urgente: cancellarle.

6.2 JSON della lezione (`edu_assembla_lezione`)

```
{ "id":"salinaio", "uuid":"…", "em":"🧂", "cls":"rosa", "tit":"…", "sub":"…", "durata":"12 min",
  "intro":"…", "prologo":null, "stato":"pubblicata", "coccio":"<video mascotte apertura>",
  "acc":{ "ipo":true, "lis":true, "lis_clip":3, "n_clip":3 },
  "video":[ { "tipo":"video", "guid":"<bunny>", "url":null, "durata_s":138, "lis":"<guid LIS>",
              "audio":{ "url":"https://…/a.mp3", "vol":45, "loop":true, "duck":true } } ],
  "tappe":[ { "v":0, "t":41, "cocciu":"…", "cocciu2":"…",
              "hotspots":[ { "yaw":-157, "pitch":-9, "em":"🧂", "tit":"…", "txt":"…", "txt2":"…", "foto":"…", "audio":"…" } ] } ],
  "quiz":[ { "q":"…", "opts":["…"], "ok":0, "spiegazione":"…", "media_spiegazione":"…", "v":0 } ],
  "quiz2":[ … ],
  "corso":{ "id":"la-salina", "uuid":"…", "tit":"…", "prologo":"…", "mascotte":{ "nome":"…", "img":"…", "junior":true, "senior":true, "quiz_sempre":true, "video":{ "entrambi":
{ "inizio_corso":"bunny:…" }, "junior":{}, "senior":{} } } } }
```

Convenzioni: `yaw` 0 = davanti, 90 = destra, 180 = dietro; `pitch` positivo = alto. Nel player la posizione 3D è `x=r·sin(yaw)·cos(pitch)`, `y=r·sin(pitch)`, `z=−r·cos(yaw)·cos(pitch)` con la videosfera ruotata `0 -90 0`. **Attenzione**: `normalizzaLezione()` in `index.php` ricostruisce ogni clip conservando solo i campi elencati (`lis`, `audio`): un campo nuovo va aggiunto lì o non arriva al player (bug pagato due volte).

6.3 `live.php` (quiz live)

Stato della stanza in `dati/live/{pin}.json`: `{pin, host, titolo, quiz:[{domanda, opzioni, g, spiegazione, media}], stato:'attesa'|'domanda'|'risultati'|'fine', q_idx, giocatori:{token:{nome, risposte}}, podio, creato}`. Azioni (JSON): `crea{titolo, quiz, podio}`→`{pin, host}`; `join{pin, nome}`→`{token, nome}` (nome ripulito, max 200 giocatori); `stato?pin&token` (l'host vede i nomi e il recap, l'alunno solo domanda/opzioni; `giusta` e spiegazione **solo** nello stato `risultati`); `risposta{pin, token, q, r}` (una sola per domanda: la seconda risponde `gia:true`); `avanza{pin, host}` (attesa→domanda→risultati→…→fine); `rimuovi`, `chiudi`, `ver`. Throttle per IP e per azione: `stato` 20 000/min, `risposta` 6 000/min, resto 1 200/min, `join` 500/5 min, `crea` 20/10 min. Stanze

TTL 3 h. Stress: 500 alunni in 10 stanze senza errori.

6.4 `stripe.php` e webhook

`POST stripe.php?azione=crea|pausa|riprendi|annulla|stato|link` con `Authorization: Bearer <access_token Supabase>` e corpo JSON `{cliente_id, piano?, periodo?, quando?}` → `{ok, messaggio, stato, link, …}`. Il webhook riceve gli eventi Stripe e li normalizza in `{id, tipo, subscription, customer, invoice, importo_cent, valuta, periodo_da, periodo_a, stato_sub, pausa, cancel_at_period_end, link, stato_invoice, cliente_id}` per la RPC.

7. Sicurezza — modello e meccanismi

- **Superficie anonima (chiave pubblica nel browser di chiunque)**: nessuna lettura di tabelle (`revoke` generale, migr. 27/29); solo `INSERT` su `edu_eventi` (tipo e codice controllati dalla policy) e `edu_richieste`; RPC pubbliche di sola lettura elencate in §5.2. Verificato con una matrice ruoli × tabelle × operazioni (`t_rls_matrice.sql`).
- **RLS**: set canonico (migr. 29): per ogni tabella al più una policy per (ruolo, operazione) — `<tab>_sel` e `<tab>_wr`; un controllo finale della migrazione fallisce se trova doppioni. Le policy non fanno sotto-query incrociate fra tabelle protette (ricorsione infinita): usano i helper `SECURITY DEFINER`. Regole: docente legge il proprio cliente, scrive solo le proprie classi; editor scrive corsi/lezioni/scuole/classi del proprio cliente e concede licenze solo dei propri corsi; admin tutto; i corsi in licenza sono in sola lettura per il licenziatario.
- **Player/API**: token di sessione obbligatorio (`X-EDU-T`), referer, throttle per IP (contatori su file, condivisi), contenuti cifrati per sessione, gate licenza a tre livelli, `dati/` negata dal web, cache mai servita direttamente.
- **Regia**: `esc()` scappa `& < > " '` (i nomi finiscono in `onclick`); slug riservati; editor senza corso non può creare lezioni ufficiali; nessuna chiave service nel browser.
- **Stripe**: admin verificato server-side; firma HMAC con tolleranza 5 min; RPC riservata al service role; idempotenza per evento.
- **Account**: attivare MFA (TOTP) sull'admin; rimuovere/declassare gli utenti di prova.

8. Operatività

8.1 Deploy di una modifica

1. Test locali verdi (§9). 2. Aggiornare il badge di versione (regia: `vNN · nome`; api: stringa in `?ver`; player: commento in testa) — è l'unico modo per capire dal browser cosa è in produzione. 3. Migrazioni su Supabase (SQL Editor, file idempotente). 4. FTP in `/project/edu/app/` (FileZilla in modalità ASCII converte i fine-riga: innocuo, `debug500` lo tollera). 5. Cancellare `dati/cache_*.json`. 6. Verifica: `api.php?ver`, `debug500.php`,

`/edu/salina-culcasi/`, regia (badge), un hotspot piazzato, un quiz live con due telefoni.

8.2 Stripe

Vedi `LEGGIMI-FASE2-STRIPE.md`: chiavi test → `migrazione_30` → file → config → `stripe-setup.php` (poi cancellarlo) → endpoint webhook + `whsec` → «Email finalized invoices» → prova con carta `4242…` → chiavi live e nuovi price ID. Diagnostica: dashboard Stripe → Webhooks → eventi (risposte 200/400/500 del nostro endpoint), `edu_pagamenti`.

8.3 Manutenzione

- `dati/log.txt` cresce: ruotarlo. `dati/rl/` si autopulisce (1/50 richieste). `dati/live/` TTL 3 h.
- Backup: Supabase (Database → Backups) + copia periodica di `dati/config.php`.
- Nuovo cliente: regia → Clienti → nuovo (o «Crea cliente» da una richiesta) → licenza → utenti → corsi (propri o dal catalogo) → scuole/classi → link alla scuola. Il flusso è coperto da `t_flusso_tenant.sql` e `t_flusso_regia.js`.

9. Test — come si eseguono

Cartella dello zip (`esplora-trapani-edu-CATALOGO.zip`). Prerequisiti sandbox: `node` + `npm i jsdom`, `php-cli` + `php-curl`, `postgresql`, `apache2` con `mod_rewrite` e `mod_php` (per i test con `.htaccess`).

Suite	Comando	Copre
Player	`node test-profondo.js`, `node t_player26.js`, `node t_audio.js` (servono le pagine renderizzate da PHP/Apache in `/tmp/pag*.html`)	menu, prologo, licenze, accessibilità, quiz, audio, mascotte del corso
Regia	`node test-regia.js`, `node test-regia26.js`, `node t_flusso_regia.js`, `node t_mascotte.js`, `node t_audit.js`, `node t_viewer.js`, `node t_regia_abbonamenti.js`, `node t_sec_xss.js`, `node t_regia_audio.js`	tutti i pannelli con Supabase mockato (fixture in-memory), XSS, viewer video, Stripe UI
Statica	`node t_statica.js`	ogni funzione chiamata negli script deve essere definita
DB	`psql -f migrazione_completa.sql` su replica (`deploy/replica.sql`), `t_rls_matrice.sql`, `t_flusso_tenant.sql`, `t_abbonamenti.sql`	migrazioni, RLS, flusso tenant, abbonamenti
API	`bash t_api_flussi.sh` (Apache), `bash t_api26.sh`	sessione, cifratura, referer,

Suite	Comando	Copre
		licenze, cache
Live	`bash t_live_flussi.sh`, `node stress-live.js N stanze secondi`	casi limite e carico
Stripe	`bash t_stripe.sh` con `mock/stripe-mock.php` e `mock/supa-mock.php` (`php -S` su 8091/8092/8090)	comandi, webhook, setup

Regole dei test (pagate care): **mai stubbare una funzione dell'app per far passare un test** (se non esiste in jsdom, non esiste nel browser); il viewer si esercita davvero; python `open()` converte i CRLF — usare `newline=''` per patchare `index.php`/regia.

10. Come si aggiunge una funzionalità — tre percorsi guidati

10.1 Un campo nuovo sulla clip (es. `sottotitoli_url`)

1. **Migrazione** `migrazione_31_*.sql`: `alter table edu_media add column if not exists sottotitoli_url text;` e **ridefinire** `edu_assembla_lezione` aggiungendo il campo nel `jsonb_build_object` di `video` (copiare l'ultima definizione dalla 25). Provare sulla replica.
2. **api.php**: passa tutto ciò che la RPC restituisce — nessuna modifica, ma cancellare le cache.
3. **index.php**: in `normalizzaLezione()` aggiungere il campo al `Object.assign(B(v.guid), {lis:…, audio:…, sottotitoli:v.sottotitoli_url||null})`; poi usarlo dove serve (`attach()`).
4. **Regia**: in `renderMedia()` aggiungere il campo nella riga della clip e una `salvaX(id)` che fa `sb.from('edu_media').update({...}).eq('id',id)` con validazione URL (vedi `salvaAudio`).
5. **Test**: estendere `t_audio.js`/`t_regia_audio.js` (pattern: menu/lezione mockati, asserzioni sul DOM e sulle scritture), rieseguire `t_statica.js`.

10.2 Un pannello nuovo in regia (es. «Comunicazioni ai docenti»)

1. Tabella + RLS nella migrazione: policy `<tab>_sel` / `<tab>_wr` con i helper (`edu_e_admin()`, `edu_mio_cliente()`, `edu_sono_editor()`); `revoke all … from anon`.
2. Markup: un `<div id="xPanel" class="hidden">` nella colonna centrale; pulsante in sidebar; aggiungere l'id a `nascondiTutto()`.
3. Funzioni `apriX()` (chiama `caricaTenant()`, `nascondiTutto()`, mostra), `renderX()`, `salvaX()` con `esc()` su ogni testo e id — mai nomi dentro `onclick` senza `esc`.
4. Test jsdom sul modello di `test-regia26.js` (fixture `window.__fx`, mock `from()` con `then`).

10.3 Un limite di piano nuovo (es. numero massimo di lezioni)

1. `edu_piano_limiti` (migr. 24): aggiungere la chiave al jsonb per ogni piano.
2. Regia: verificare prima dell'insert (vedi `corsoPubblicabile`/`nuovaClasse`) con `sb.rpc('edu_piano_limiti', {p_piano})` e un toast che spiega cosa fare.
3. Player (se il limite cambia l'esperienza): il menu porta `cliente.limiti` — usarlo come `QUIZ_LIVE_OK`.
4. Test: caso "limite raggiunto" e "piano superiore consentito" (vedi `test-regia.js`, sezione piani).

10.4 Regole trasversali

- Ogni RPC pubblica è `SECURITY DEFINER` con `set search_path to 'public'` e grant esplicito (`anon` solo se serve al player).
- Ogni scrittura di sistema (webhook, cron) passa da una RPC riservata a `service_role`, mai da grant su tabelle.
- Aggiornare il badge di versione e i LEGGIMI; la skill/documentazione vive nello zip.

11. Trappole note (leggere prima di toccare)

1. `normalizzaLezione` scarta i campi non elencati. 2. RLS: policy che si interrogano a vicenda = ricorsione infinita → helper SECURITY DEFINER. 3. Supabase annulla l'intero file SQL al primo errore. 4. `edu_utenti` non ha `email`; `edu_eventi` ha `ts` e `creato_il`. 5. `edu_classi.codice` è la PK storica; `id` è unico ma aggiunto dopo. 6. Bunny: una sola libreria; referrer; con `same-origin` 403. 7. Aruba: permessi 644/755, `exec` no, `mbstring` non garantito, cache 5 min. 8. FileZilla ASCII converte CRLF (innocuo). 9. Barra finale negli URL tenant (forzata dall'`.htaccess`; il player usa `EDU_BASE`). 10. Gmail pre-apre i link di recupero password: usare un link solo, o reimpostare via SQL `crypt(...)`. 11. Il quiz live è su file: non scalare oltre un server senza spostarlo su DB. 12. La demo VR (`demo-vr-quest.zip`) è separata: nel Quest il video era nero, causa da confermare con `prova-video.html` (referrer/CORS).

12. Mappa dei file consegnati

- `app/` — codice di produzione (§4). `htaccess-project-edu.txt` → `/project/edu/.htaccess`.
- `migrazione_completa.sql` (19→30) e i singoli `migrazione_NN_*.sql` (documentazione).
- `mock/` — finto Stripe e finto Supabase per i test. `deploy/replica.sql` (nella cartella di lavoro) — replica dello schema reale.
- Test: `test-*.js`, `t_*.js`, `t_*.sql`, `t_*.sh`, `stress-live.js`.
- Documenti: `LEGGIMI-CATALOGO.md` (deploy + relazione dei test + audit), `AUDIT-FASE1.md`, `LEGGIMI-FASE2-STRIPE.md`, `GUIDA-AUDIO-GOCCIA.md`, `ARCHITETTURA-EDU-SAAS.md`, `MANUALE-QUIZ-

`LIVE.pdf`, `LEGGIMI-FASE2/3/4.md` (storici).

- Fuori dallo zip: `demo-vr-quest.zip` (demo Meta Quest), relazione per il progettista di finanza agevolata (docx/pdf).